A Game-Theoretic Analysis of MEV in Decentralised Exchanges

Progress Report 2

Name	Roll Number
Sanoj S Vijendra	22B0916

March 25, 2026

Abstract

The report provides the theoretical background as in phase1 along with the simulation code logic for phase 2. The theoretical model, a three-player game capturing strategic interactions among users, miners/validators, and the DEX protocol is briefly recapped. This report describes the Python simulation codebase: seven interacting modules that implement a peer-to-peer blockchain network with a constant-product automated market maker (AMM) DEX, three distinct peer archetypes (honest, sandwich, front-runner), an event driven simulation engine, and supporting data structures for blocks, transactions, and the chain tree. Each module is mapped to its corresponding game theoretic concept from phase1. Empirical results and welfare comparisons are deferred to phase3.

Contents

1 Introduction **2**
1.1 Progress in This Report . . . 2

2 Theoretical Model Recap **2**
2.1 Game Structure . . . 2
2.2 AMM Mechanics . . . 3
2.3 Miner Strategies . . . 3
2.4 Payoff Functions . . . 3

3 Simulation Codebase **3**
3.1 `txn.py` — Transaction Model . . . 4
3.2 `dex.py` — Automated Market Maker . . . 4
3.3 `block.py` — Block Data Structure . . . 5
3.4 `blocktree.py` — Chain Tree and Fork Management . . . 5
3.5 `peer.py` — Peer / Node Model . . . 6
3.6 `blockchain.py` — Network Topology and State . . . 6
3.7 `simulation.py` — Event-Driven Simulation Engine . . . 7
3.7.1 Initialisation: `create_blockchain` . . . 7
3.7.2 Latency Model . . . 7
3.7.3 Event Types and Handlers . . . 8
3.7.4 Transaction Validation . . . 8

4 Mapping: Code to Game-Theoretic Model **9**

5 Next Steps **10**
5.1 MEV Strategy Logic . . . 10
5.2 DEX Integration with Block Execution . . . 10
5.3 Batch Auction Clearing . . . 10
5.4 Metrics Collection . . . 10

6 Plan for Report 3 **10**

7 Conclusion **11**

1 Introduction

Decentralised exchanges (DEXs) built on public blockchains expose the full transaction lifecycle, from submission to the peer-to-peer mempool through to on-chain settlement to sophisticated rational agents. *Maximal Extractable Value* (MEV) refers to the surplus value that miners or validators can capture by selectively reordering, inserting, or censoring transactions within a block. Its most studied manifestations are **frontrunning**, **backrunning**, and **sandwich attacks**, all of which transfer welfare from ordinary users to block producers and arbitrage bots.

Report 1 contained the theoretical foundations: a literature review of the MEV landscape and a formal three-player Stackelberg game model. This report builds on that foundation by describing the Python simulation codebase. The codebase models a a blockchain network with an embedded constant-product AMM DEX and three strategic peer types, enabling the empirical measurement of user welfare, miner revenue, and MEV extraction under different protocol configurations.

1.1 Progress in This Report

- A condensed recap of the formal model from Report 1 (§2).
- A module-by-module description of the simulation codebase (§3), covering `txn.py`, `dex.py`, `block.py`, `blocktree.py`, `peer.py`, `blockchain.py`, and `simulation.py`.
- A mapping between code constructs and the game theoretic abstractions defined in Report 1 (§4).
- A discussion of what remains to be implemented and the plan for Report 3 (§6).

2 Theoretical Model Recap

This section gives a concise recap of the model from Report 1.

2.1 Game Structure

The DEX ecosystem is modelled as a **three player Stackelberg game** with the following sequential stages:

Stage 1. User stage. Users observe the current AMM state (r_A, r_B) and submit signed transactions specifying a gas price g_i, trade size δ_i, and slippage tolerance τ_i^{set}.

Stage 2. Miner stage. The block producer observes the full mempool $\mathcal{P}$ and selects an ordering σ together with a (possibly empty) set of own-inserted transactions $\mathcal{I}$ to maximise revenue.

Stage 3. DEX settlement stage. Transactions are cleared according to the protocol rule (continuous AMM or batch auction).

Users are *leaders* who commit before the miner's ordering decision; miners are *followers* who best-respond to the observable mempool.

2.2 AMM Mechanics

The constant-product AMM maintains reserves (r_A, r_B) subject to the invariant $r_A \cdot r_B = k$. A swap of Δ_A units of token A for token B yields:

$$\Delta_B = \frac{r_B \cdot \Delta_A}{r_A + \Delta_A}.$$

The spot price is $P(r_A, r_B) = r_B / r_A$.

2.3 Miner Strategies

Three canonical strategies are modelled:

M1. Honest ordering: transactions ordered by descending gas price; no own insertions.

M2. Frontrunning: miner inserts own transaction(s) before a detected profitable user trade.

M3. Sandwich attack: miner inserts a buy transaction before and a sell transaction after a large user trade, profiting from the induced price impact:

$$\pi_M^{\text{sandwich}} = \Delta_B^{y^+} - \Delta_B^{y^-} > 0.$$

2.4 Payoff Functions

The user payoff is:

$$\pi_i^U = v_i \cdot \frac{\hat{\Delta}_B^i}{\Delta_B^i} - g_i \cdot c_{\text{gas}} - \mathbb{1}[\text{reverted}] \cdot g_i^{\text{used}} \cdot c_{\text{gas}},$$

where $\hat{\Delta}_B^i$ is the actual output and Δ_B^i the fair (no-MEV) output. The miner payoff is:

$$\pi_M = \sum_{i=1}^{k} g_{\sigma(i)} \cdot \text{gas}_{\sigma(i)} + \sum_{y_j \in \mathcal{I}} \pi_M^{(j)} + B_{\text{block}}.$$

Social welfare is $W = \sum_i \pi_i^U + \pi_M - \text{DWL}$, where DWL is the deadweight loss from reverted transactions and PGA gas waste.

3 Simulation Codebase

The simulation is implemented in seven Python modules. The overall architecture is shown schematically below:

Module Dependency Overview

`simulation.py` $\longrightarrow$ `blockchain.py` $\longrightarrow$ `peer.py`, `dex.py`

`peer.py` $\longrightarrow$ `blocktree.py`, `block.py`

`block.py` $\longrightarrow$ `txn.py`

Each module is described in detail below.

3.1 `txn.py` — Transaction Model

Purpose. Defines the `Transaction` class, the atomic unit of user intent submitted to the mempool.

Key attributes and design.

- `type`: either `"swap"` (DEX trade) or `"transfer"` (token transfer between peers).
- **Swap parameters**: `dex_id`, `idx1`, `idx2` (token indices), `amt` (input amount), `tlr` (slippage tolerance τ_i^{set}), and `min_out` (minimum acceptable output, encoding the user's revert condition).
- **Transfer parameters**: `idx` (token), `recv` (receiver peer id), `amt`.
- `gas_price`: the gas price bid g_i attached to the transaction, used by miners to prioritise ordering.
- `timestamp`: simulation time of creation.
- `txn_id`: randomly assigned unique identifier.
- `code`: a randomly assigned integer (placeholder for smart-contract bytecode complexity, related to gas consumption).

Design note. The separation of `"swap"` and `"transfer"` types reflects the real-world distinction between DEX interactions and simple ERC-20 transfers. Only swap transactions are relevant to MEV analysis; transfer transactions serve as background traffic in the simulation.

3.2 `dex.py` — Automated Market Maker

Purpose. Implements the constant-product AMM as described in first report.

Key methods.

- `init_tokens(idx, val)`: initialises the reserve $r_{\texttt{idx}}$ of token `idx` to `val`, establishing the initial state (r_A, r_B) of the AMM.
- `get_relative_price(idx1, idx2)`: computes the spot price $P = r_{\texttt{idx2}}/r_{\texttt{idx1}}$.
- `swap(idx1, idx2, amt)`: executes the constant-product swap formula. Given input $\Delta = \texttt{amt}$ of `idx1`, the method computes:

$$\texttt{dec} = \frac{r_{\texttt{idx2}} \cdot \texttt{amt}}{r_{\texttt{idx1}} + \texttt{amt}}$$

  updates the reserves, and returns the output Δ_B.
- `get_curr_state()`: returns the current reserve vector, enabling the simulation to record AMM state evolution across blocks.

Design note. Each `Blockchain` instance holds a list of `DEX` objects, supporting multi-DEX simulations in future (if significant progress is made and time permits). Currently, a single two-token DEX is initialised. The implementation is stateful: every `swap` call permanently updates reserves, so the order of transaction execution directly determines the price path—the core source of MEV in the model.

3.3 `block.py` — Block Data Structure

Purpose. Defines the `Block` class, representing a single block on the blockchain.

Key attributes.

- `id`: a randomly generated 64-bit integer serving as the block's unique hash.
- `prev_blk_id`: the identifier of the parent block, forming the chain linkage.
- `txns`: the ordered list of `Transaction` objects included in the block. The ordering reflects the miner's chosen permutation σ plus any inserted transactions $\mathcal{I}$.
- `creator_id`: the peer id of the miner who produced the block.
- `balance`: a dictionary mapping each peer id to a list of per-token balances *after* applying the block's transactions. This snapshot is used for validation by receiving peers.
- `time`: the simulation timestamp at which mining completed.

Design note. The `balance` snapshot per block enables efficient validation without replaying the entire chain history: a receiving peer need only check the *parent block's* balance snapshot against the candidate block's transactions. This mirrors the UTXO/account model balance checking done by real nodes.

3.4 `blocktree.py` — Chain Tree and Fork Management

Purpose. Implements the `BlockTree` class, which maintains each peer's local view of the blockchain including forks.

Key data structures.

- `tree`: a list of dictionaries, where `tree[level]` maps each block id at that chain depth to its parent id. Level 0 holds the genesis block.
- `id_level_map`: maps block ids to their depth, enabling $O(1)$ level lookup.
- `orphan_blocks`: a dictionary of blocks received before their parent. Upon adding a block, the tree checks whether any orphan's parent has now been added and resolves the dependency recursively.

Key methods.

- `add_block(id, prev_blk_id)`: inserts a new block at depth $d_{\text{parent}} + 1$. If the parent is not yet known, the block is quarantined in `orphan_blocks`.
- `get_longest_chain_leaves()`: returns all block ids at the deepest level of `tree`, i.e. the tips of all candidate longest chains. When multiple leaves exist, the simulation resolves ties by selecting the one with the earliest timestamp (earliest-seen rule).
- `get_active_blocks()`: counts all blocks that lie on at least one path from the deepest level back to the genesis, providing a measure of chain utilisation vs. stale (orphaned) blocks.

Design note. Maintaining the full tree (rather than only the longest chain) is essential for capturing fork dynamics. Forks arise naturally in the simulation due to propagation

latency, and the frequency and depth of forks under different peer configurations is a key output metric.

3.5 `peer.py` — Peer / Node Model

Purpose. Defines the `Peer` class, representing a single participant in the blockchain network. Each peer maintains its own local state: a mempool, a block tree, and type-specific behaviour parameters.

Key attributes.

- `peer_id`: unique integer identifier.
- `type`: one of `"user"`, `"honest"`, `"sandwich"`, or `"front-runner"`, mapping directly to the player taxonomy in of Report 1.
- `latency`: one of `"low"`, `"mid"`, `"high"`, determining the peer's effective bandwidth and propagation delay.
- `mempool`: a list of pending `Transaction` objects not yet included in a block.
- `mempool_txn_id_map`: a dictionary for $O(1)$ duplicate detection during gossip propagation.
- `gen`: the genesis `Block` for this peer's chain, initialised with a sentinel id of -2.
- `tree`: the peer's local `BlockTree` instance.
- `prev_blk`: the id of the block the peer is currently mining on top of.

Peer type mapping. The four peer types correspond to the model players as follows:

Code type	Model player	Strategy
`"user"/"bots"`	Player 1 ($\mathcal{U}$)	Passive transaction generator/Arbitrage Seeker
`"honest"`	Player 2 (M), strategy M1	Orders by gas price; $\mathcal{I} = \emptyset$
`"sandwich"`	Player 2 (M), strategy M3	Inserts sandwich around user swaps
`"front-runner"`	Player 2 (M), strategy M2	Inserts own txn before profitable swaps

Table 1: Mapping of `Peer.type` to game-theoretic players and miner strategies.

Design note. All three miner archetypes (`"honest"`, `"sandwich"`, `"front-runner"`) are present simultaneously in every simulation run, competing to mine blocks. This reflects the heterogeneous miner population observed on real networks, where different validators adopt different MEV strategies.

3.6 `blockchain.py` — Network Topology and State

Purpose. The `Blockchain` class serves as the global state container for the simulation: it holds the list of all peers, the network topology, and the DEX instances.

Key methods.

- `create_topology()`: generates a random peer-to-peer network graph. Each peer is assigned between 3 and 6 neighbours, subject to the constraint that the graph is connected (verified via DFS). If connectivity fails, the topology is regenerated from scratch. This mirrors the gossip-network model implicit in the latency analysis of Daian et al. [1].
- `add_dex(dex)`: registers a `DEX` instance with the blockchain, allowing multiple AMMs to coexist.
- `get_peer(id)`: linear-scan lookup returning the `Peer` object with the given id.

Topology design. The random topology with degree bounds $[3, 6]$ produces a realistic small-world-like network. Propagation latency is a function of both the network topology (hop count) and per-link bandwidth, consistent with the empirical latency model of Daian et al. where mean inter-bid times are a proxy for message propagation speed.

3.7 `simulation.py` — Event-Driven Simulation Engine

Purpose. The top-level module that initialises the network and drives the simulation forward through a discrete-event loop.

3.7.1 Initialisation: `create_blockchain`

The `create_blockchain` function:

1. Creates n `user` peers partitioned by latency class according to fractions z_0 (low), z_1 (mid), z_2 (high).
2. Appends three fixed special peers: one `"honest"` miner, one `"sandwich"` miner, and one `"front-runner"` miner—all assigned low latency to reflect that MEV-extracting nodes typically invest in fast infrastructure.
3. Calls `create_topology()` to wire up the P2P graph.
4. Initialises a single two-token DEX with reserves $r_A = t_A$ and $r_B = t_B$ (user-supplied).
5. Distributes initial token balances uniformly across all user peers: each peer receives $t_A/(10n)$ of token A and $t_B/(10n)$ of token B.

3.7.2 Latency Model

The `latency(i, j, m)` function computes the propagation delay (in milliseconds) for a message of size m bits between peers i and j:

$$\ell(i, j, m) = p + \frac{m}{c} + d, \qquad p \sim \text{Uniform}(10, 500)\,\text{ms}, \quad d \sim \text{Exp}\left(\frac{96 \times 1024 \times 10^3}{c}\right),$$

where c is the link capacity determined by the *slower* of the two peers' latency classes:

$$c = \begin{cases} 0.05 \times 2^{20}\,\text{bit/s} & \text{if either peer is “high” latency,} \\ 5 \times 2^{20}\,\text{bit/s} & \text{if either peer is “mid” latency,} \\ 100 \times 2^{20}\,\text{bit/s} & \text{otherwise (both “low”).} \end{cases}$$

This bottleneck model is consistent with the empirical observation that propagation latency in P2P networks is dominated by the slowest link on the path.

3.7.3 Event Types and Handlers

The simulation maintains a min-heap priority queue (`heapq`) over events sorted by scheduled time. Five event types are processed:

Event type	Description
`GENERATE_TRANSACTION`	A peer creates a new `Transaction`, adds it to its local mempool, and gossips it to all neighbours with per-link delay. The next transaction is rescheduled with an $\mathrm{Exp}(T_{\mathrm{tx}})$ delay.
`RECEIVE_TRANSACTION`	A peer receives a gossiped transaction. Duplicate detection (via `mempool_txn_id_map`) prevents re-broadcasting already-known transactions. New transactions are forwarded to all neighbours except the sender.
`START_MINING`	A peer selects the tip of its longest chain (earliest timestamp on tie-break) and schedules a `FINISH_MINING` event after an $\mathrm{Exp}(I/h_i)$ delay, where I is the target mean inter-block time and h_i is the peer's relative hash power.
`FINISH_MINING`	Stale if the mined-on block is no longer the chain tip. Otherwise, selects up to 124 transactions from the mempool (ordered by timestamp), validates balances, appends a coinbase transaction, broadcasts the new block to neighbours, and restarts mining.
`RECEIVE_BLOCK`	Validates all transactions in the received block against the parent block's balance snapshot. On success, removes included transactions from the mempool, extends the local `BlockTree`, and restarts mining if the longest chain has grown.

Table 2: Simulation event types and their handlers.

3.7.4 Transaction Validation

On `FINISH_MINING` and `RECEIVE_BLOCK`, each transaction is validated against a running balance snapshot initialised from the parent block. Invalid transactions (insufficient balance) are skipped or cause block rejection respectively. This implements the basic double-spend prevention of the model.

4 Mapping: Code to Game-Theoretic Model

Table 3 provides a systematic mapping between the code constructs and the formal model elements defined in Report 1.

Table 3: Mapping between simulation code and game-theoretic model.

Code construct	Model element	Notes
`DEX.swap()`	AMM output formula	Implements $\Delta_B = r_B\Delta_A/(r_A + \Delta_A)$ exactly.
`Transaction.gas_price`	User strategy g_i	Gas price bid determining mempool priority.
`Transaction.tlr`, `min_out`	Slippage tolerance τ_i^{set}	Encodes the user's revert condition for sandwich protection.
`Peer.type = "sandwich"`	Miner strategy M3	Will insert front and back legs around large swaps.
`Peer.type = "front-runner"`	Miner strategy M2	Will insert own trade before detected profitable swaps.
`Peer.type = "honest"`	Miner strategy M1	Orders by gas price; no own insertions.
`Peer.mempool`	Mempool $\mathcal{P}$	Full set of pending transactions observable by the miner.
`BlockTree`	Blockchain history / fork structure	Tracks all candidate chains; longest-chain rule implemented.
`Block.txns` ordering	Permutation σ	Currently honest (timestamp order); MEV strategies will reorder.
`Block.balance`	Running balance snapshot	Enables efficient validation of π_i^U (balance checks).
`latency(i,j,m)`	Latency Δ_i^U (Def. 3.11)	Models information delay; affects block propagation and fork rate.
`Simulator.event_queue`	Game timeline / block arrival process	Exponential block times implement Poisson mining.

5 Next Steps

While the blockchain infrastructure and basic transaction flow are implemented, several components required for the full game-theoretic simulation are not yet complete.

5.1 MEV Strategy Logic

The most significant absence is the actual MEV strategy implementations for the `"sandwich"` and `"front-runner"` peer types. Currently, all three miner types behave identically in the `FINISH_MINING` handler (honest timestamp ordering, no insertions). To be added:

- **Sandwich miner**: on `FINISH_MINING`, scan the mempool for large swap transactions where $\pi_M^{\text{sandwich}}(\delta_i) >$ gas cost; if found, construct and prepend a front-leg transaction y^+ and append a back-leg transaction y^- to the block.
- **Front-runner**: scan the mempool for any swap that produces a price movement; insert an identical or larger swap ahead of it at a higher gas price.

5.2 DEX Integration with Block Execution

Currently, the `DEX` object is initialised at the blockchain level but is not invoked during block execution. The `FINISH_MINING` handler processes only balance transfers, not AMM swaps. Adding swap execution requires:

- Routing `Transaction.type == "swap"` through `DEX.swap()` in the correct sequence during block construction and validation.
- Maintaining a per-block DEX state snapshot (analogous to `Block.balance`) for validation by receiving peers.

5.3 Batch Auction Clearing

The batch auction mechanism is not yet implemented. This is the primary comparison protocol and will be added as a configurable clearing mode in the `Blockchain/Simulator` layer.

5.4 Metrics Collection

No welfare metrics are currently recorded. The following will be added:

- Per-user slippage slip_i across all executed swaps.
- Miner MEV revenue $\sum_{y_j \in \mathcal{I}} \pi_M^{(j)}$ per block.
- Gas fee waste (transactions reverted due to slippage violations).
- DEX reserve evolution $(r_A(t), r_B(t))$ and price path $P(t)$.

6 Plan for Report 3

The following items constitute the development roadmap for Phase 3:

P1. Complete DEX integration: wire `DEX.swap()` into block execution and validation for swap transactions.

P2. Implement MEV strategies: add sandwich and front-runner logic to the respective `FINISH_MINING` handlers.

P3. Implement batch auction clearing: add a `BatchAuction` clearing mode as an alternative to sequential execution.

P4. Add metrics collection: record per-block and per-simulation welfare statistics.

P5. Run experiments: simulate across a parameter grid (network size n, mean inter-block time I, MEV strategy type) and report empirical payoffs.

P6. Compare equilibrium outcomes: contrast continuous AMM and batch-auction welfare metrics to validate the theoretical predictions as in first report.

P7. Attack Analysis: analysis of different attacks possible due to fake timestamp in transactions.

7 Conclusion

The codebase implements a complete layer-1 blockchain network with peer-to-peer gossip propagation, fork resolution, and a constant-product AMM DEX. Three peer archetypes—`honest`, `sandwich`, and `front-runner`—are defined and structurally mapped to the miner strategies of the game-theoretic model from Report 1. The event-driven simulation engine faithfully captures the sequential structure of the Stackelberg game: users submit transactions to the mempool, miners select orderings and insert own transactions, and the DEX settles the resulting state transitions.

The principal remaining tasks are the implementation of the actual MEV strategy logic, full DEX integration into block execution, and the batch auction clearing mode. These will be completed in Phase 3, at which point empirical welfare comparisons between continuous AMM and batch-clearing protocols will be reported.

References

[1] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralised exchanges. *arXiv preprint arXiv:1904.05234*, 2019.

[2] Zeinab Alipanahloo, Kaiwen Zhang, and Emmanuel Awosika. Quantifying fairness granularity as a fair ordering policy towards MEV mitigation for rollups. In *2024 IEEE International Conference on Blockchain (Blockchain)*, pages 209–218. IEEE, 2024.

[3] Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

[4] Miles Carlsten, Harry Kalodner, S. Matthew Weinberg, and Arvind Narayanan. On the instability of bitcoin without the block reward. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 154–167, 2016.

[5] Martin Shubik. The dollar auction game: A paradox in noncooperative behavior and escalation. *Journal of Conflict Resolution*, 15(1):109–111, 1971.